



Second Semester 2025-26

Course Handout Part II

Date: Jan 5, 2026 | Last updated: Jan 16, 2026

In addition to [Part I](#) (the General Handout for all courses, appended to the Timetable), this section provides further details specific to the course.

Course Number	: CSF314
Course Title	: Software Development for Portable Devices
Instructor-in-charge	: Swaroop Ravindra Joshi (swaroopj)
TAs	: Saumya Mathkar (p20210017)
Class details	: W 11-1, F 11-12 C301

Course description

(As per the bulletin) Introduction to mobile computing and emerging mobile application and hardware platforms; Developing and assessing mobile applications; Software lifecycle for mobile application – design and architecture, development – tools, techniques, frameworks, deployment; Human factors and emerging human computer interfaces (tangible, immersive, attentive, gesture, zero-input); Select application domains such as pervasive health care, m-Health; Mobile web browsing, gaming and social networking.

Scope and Objective

This elective provides a comprehensive introduction to mobile computing, emerging platforms, and the full software lifecycle for mobile application development—from requirements analysis through design, architecture, development (including tools, techniques, and frameworks), deployment, and maintenance. The course emphasizes cross-platform development (Android/Kotlin, iOS/Swift, Flutter), architectural design patterns, non-functional requirements, and user experience constraints unique to portable devices (smartphones, tablets, wearables). Students are encouraged to explore human factors and emerging human-computer interfaces (tangible, gesture, zero-input), with hands-on work assessing and developing mobile applications through project-based learning through team-based projects using agile methodologies to simulate industry workflows.

Course learning outcomes

Upon completion of the course, a student will be able to:

1. Apply an industry-standard software development process to plan, track, and deliver increments of a real-world software system, including documentation of process decisions.
2. Integrate various hardware sensors (e.g., location and gesture detection) and out-of-the-box software

services (e.g., notifications) available on portable devices (such as mobile phones, tablets, and smartwatches) into their software products.

3. Design and model software architectures and components (e.g., using UML) that address functional and quality requirements such as scalability, maintainability, and usability
4. Implement, integrate, and maintain a multi-module software system using appropriate tools for version control, continuous integration, and dependency management.
5. Apply software quality practices, including unit/integration testing, code review, and basic verification/validation techniques, to improve product reliability.
6. Collaborate effectively in a small development team by defining roles, coordinating work, resolving conflicts, and communicating with external stakeholders.
7. Reflect on and improve team processes through regular retrospectives, connecting course experience with professional software engineering practices and ethics.

Textbooks and references

There is no textbook for this course. All the study material is available online. Some reference books/resources are listed below:

- Kristin Marsicano, Chris Stewart, and Bill Phillips, *Android Programming: The Big Nerd Ranch Guide*, Big Nerd Ranch LLC, 3rd ed., 2017
- Chris King, Robi Sen, W. Frank Ableson, *Android in Action*, Manning Publications, 2nd ed., 2011
- Valentino Lee, Heather Schneider, and Robbie Schell, *Mobile Applications: Architecture, Design and Development*, Prentice-Hall, 2004
- Raoul-Gabriel Urma, Mario Fusco, and Alan Mycroft, *Java 8 in Action: Lambdas, Streams, and Functional-Style Programming*, Manning Publications, 2014
- Benjamin Muschko, *Gradle in Action*, Manning Publications, 2014
- Craig Larman, *Applying UML and Patterns: A Guide to Object-Oriented Analysis and Design and Iterative Development*, 3rd ed., Prentice-Hall, 2004.
- Robert C Martin, *Clean Code*, Pearson, 2012.

Method of conducting the course

The course will be conducted using the Project-based Learning (PBL) pedagogy. Students will be required to design, develop, test, and demonstrate a software product that addresses a real-world problem. Stakeholders (possibly from outside the institute) will present problem statements at the start of the course. Students will organise in groups of 4 each and work on the software development from requirements gathering to final delivery over the course of the semester. The instructor will act as a facilitator, rather than a traditional teacher. Students are expected to demonstrate weekly progress through agile artifacts, such as Kanban boards.

Electronic resources

- LMS: We will use Quanta-aws only for posting grades.
- GitHub: Projects will be managed via GitHub. Regular updates to the repos are expected. We will also use it as the primary communication channel (announcements, discussions, answering queries, etc.). Students are required to check it regularly.
- Email: Your questions are more likely to be answered on GitHub discussions than in a personal email. However, please include “CS F314” in the subject line if you choose to email me. Don’t blame me if I don’t respond to an email.

Sharing and Attribution of Intellectual Property and Information

Unlike introductory CS courses, this course focuses on cutting-edge technology that evolves daily. This means there are many topics we won't be able to cover in class. Plus, you all will work on apps of your own, and a lot of "real world" code/ideas/tips, and tricks might be useful in implementing your app. Discussing and learning from each other and other resources is generally allowed for these reasons. You can exchange and use any information from each other's projects. You may also freely research and use information legally available from the Web or other sources. However, you must properly attribute each piece of borrowed intellectual property.

However, you should be able to fully explain any piece of code you have contributed to your app or any individual assignment. Failing to comply with this will be considered academic misconduct, and the student will be reported to the relevant authorities.

Course Plan (Tentative)

Week no.	Module	Lect. no.	Reference	Learning Outcome
1	Course introduction and software engineering principles	1 - 3		
2	Project kickoff	3 - 6		
3	Stakeholder needs & problem framing	6 - 9		
4	Requirements & planning	9 - 12		
5	Architecture & tech stack	12 - 15		
6	Implementation I & Dev practices	15 - 18		
7	Testing & quality	18 - 21		
8	Mid-iteration review & feedback	21 - 24		
9	Evolution & refactoring	24 - 27		
10	Advanced topics & integrations	27 - 30		
11	Hardening & validation	30 - 33		
12	Final delivery & demos	33 - 36		
13	Retrospective & reflection	36 - 39		

Evaluation Scheme

There are no written exams.

Component	Weightage	Duration	Date and time	Comments
Class participation	05	In-class	Every class	Various forms
Weekly Updates	80	Weekly	In-class	
Final presentation	15	90 min	TBA	Open to the BITS community

Criterion for NC

In accordance with the 215th Meeting of the Senate held on Monday, July 29, 2024, a student must obtain at least 40% of the class median marks to pass the course.

Chamber consultation hours

- Swaroop (D-161): MF 12:15-1:00 or via <https://calendly.com/swaroopi>
- Saumya: (D-241): Thursday, Friday, 2:00–3:00 PM. For any other time, please email P20210017@goa.bits-pilani.ac.in

Makeup Policy

There are no makeups for any component. Reasonable considerations will be given if a student group is unable to meet the initial deadlines, possibly with an impact on their marks.

Swaroop Joshi

Instructor-in-charge, CSF314

(This form must be filled, ratified by the DCA and submitted to AUGSD of the campus. Items in square brackets are to be filled as per the discretion of the IC after discussions with the team of instructors if any.)